

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.



LAB REPORT on

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

RISHITHA A K (1WA24CS231)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019 Feb-2025 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by RISHITHA A K (1WA24CS231), who is bonafide student of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS
- (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil

Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda

Professor and Head
Department of CSE
BMSCE, Bengaluru

Index

Sl. No. 1.	Experiment Title	Page No.
	a) Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (pre-emptive and non pre-emptive) b) Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) → Round Robin (Experiment with different quantum sizes for RR algorithm)	1 – 17
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	18 – 23
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms a) Rate- Monotonic b) Earliest Deadline First c) Proportional scheduling	24 - 35
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	36 - 42
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance.	43 – 45
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	46 – 50
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	51 – 56

Course Outcome

CO1	Apply the different concepts and functionalities of Operating System
CO2	Analyze various Operating system strategies and techniques
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system

Program -1

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS

b) SJF

Code:

a) FCFS

```
#include <stdio.h> int main() {
int bt[20], wt[20], tat[20], at[20];

float wtavg = 0, tatavg = 0; int n,
    printf("Enter the number of processes: "); scanf("%d", &n);
i;

    printf("Enter the arrival time and burst time for each process:\n"); for
    (i = 0; i < n; i++) { printf("Process %d - Arrival Time: ", i +
    1); scanf("%d", &at[i]); printf("Process %d - Burst Time:
    ", i + 1); scanf("%d", &bt[i]);
    } wt[0] = 0; tat[0] = bt[0]; for (i =
    1; i < n; i++) { wt[i] = wt[i - 1] +
    bt[i - 1] - at[i]; if (wt[i] < 0) wt[i] =
    0; tat[i] = wt[i] + bt[i]; wtavg
    += wt[i]; tatavg
    += tat[i];
    }          printf("\nFCFS
    Scheduling\n");
    printf("Process\tArrival
    Time\tBurst
    Time\tWaiting
    Time\tTurnaround
    Time\n"); for (i = 0; i < n;
    i++) {
```

}

Turnaround Time: %.2f\n", tatavg / n);

}

```
#include <stdio.h>
```

```
of processes: "); scanf("%d", &n);
```

```
wtavg = 0, tatavg = 0;
```

```
(i = 0; i < n; i++) {
```

%d -

$$\{ |0\rangle \quad (l = 0, | \leq 11 \rangle = \pm, \dots \quad (11) \}$$

```
i++) { for (j = i + 1; j <
```

```
n; j++) { if (bt[i] > bt[j])
```

```

        {   temp  =
            bt[i]; bt[i] =
            bt[j]; bt[j] =
            temp; temp =
            at[i]; at[i] =
            at[j]; at[j] =
            temp;
        }
    }
}

```

```

wt[0] = 0; for (i = 1; i < n; i++) { wt[i]
= wt[i - 1] + bt[i - 1] - at[i]; if
(wt[i] < 0) wt[i] = 0;
}

```

```

for (i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    wtavg    +=
        wt[i]; tatavg +=
    tat[i];
}
printf("\nSJF (Non-Preemptive) Scheduling\n"); printf("Process\tArrival
    Time\tBurst    Time\tWaiting        Time\tTurnaround Time\n"); for (i = 0; i
< n; i++) { printf("%d\t%d\t\t%d\t\t%d\t\t%d\n", i + 1,
at[i], bt[i], wt[i], tat[i]);
}

```

```
printf("\nAverage Waiting Time: %.2f\n", wtavg / n); printf("Average  
Turnaround Time: %.2f\n", tatavg / n); }
```

c) SRTF

```
#include <stdio.h>
```

```
struct Process {
```

```
    int id, at, bt, ct, tat, wt, rt, rem;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter no. of process: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for(int i = 0; i < n; i++) {
```

```
        p[i].id = i + 1;
```

```
        printf("Arrival Time of P%d: ", p[i].id);
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Burst Time of P%d: ", p[i].id);
```

```
        scanf("%d", &p[i].bt);
```

```
        p[i].rem = p[i].bt; p[i].rt = -1;
```

```
    }
```

```
    int completed = 0, time = 0, sumTAT = 0, sumWT = 0, sumRT=0;
```



```

while (completed != n) {
    int idx = -1, minRem = 1e9;

    for(int i = 0; i < n; i++) {
        if (p[i].at <= time && p[i].rem > 0) {
            if (p[i].rem < minRem) {
                minRem = p[i].rem;
                idx = i;
            }
        }
    }
}

```

```

if (idx != -1) {
    if (p[idx].rt == -1) {
        p[idx].rt = time - p[idx].at;
        sumRT += p[idx].rt;
    }
    p[idx].rem--; time++;
    if (p[idx].rem == 0) {
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
    }
}

```

```

sumTAT += p[idx].tat;

```

```

        sumWT += p[idx].wt;

        sumRT += p[idx].rt; completed++;

    }

}

else { time++; }

}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT");

for(int i=0;i<n;i++){

    printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].
    tat,p[i].wt);

}

printf("\nAvg TAT: %.2f\n", (float)sumTAT/n);

printf("Avg WT: %.2f\n", (float)sumWT/n);

printf("Avg WT: %.2f\n", (float)sumWT/n);

//printf("Avg RT: %.2f\n", (float)sumRT/n);

printf("\nAksharaKM-1BM24CS028");

return 0;

}

```

Result:

a) Fcfs

```
Burst Time of P1: 6
Arrival Time of P2: 0
Burst Time of P2: 8
Arrival Time of P3: 0
Burst Time of P3: 7
Arrival Time of P4: 0
Burst Time of P4: 3
```

Process	AT	BT	CT	TAT	WT
1	0	6	9	9	3
2	0	8	24	24	16
3	0	7	16	16	9
4	0	3	3	3	0

Average TAT: 13.00

Average WT: 7.00

b)

SJF non pre-emptive

```
Enter no. of process: 5
Arrival Time of P1: 2
Burst Time of P1: 1
Arrival Time of P2: 1
Burst Time of P2: 5
Arrival Time of P3: 4
Burst Time of P3: 1
Arrival Time of P4: 0
Burst Time of P4: 6
Arrival Time of P5: 2
Burst Time of P5: 3
```

Process	AT	BT	CT	TAT	WT
1	2	1	3	1	0
2	1	5	11	10	5
3	4	1	5	1	0
4	0	6	16	16	10
5	2	3	7	5	2

Avg WT: 3.40 2Avg TAT: 6.60

c)SJF pre-emptiv

```
Number of processes: 4

Arrival Time of P1: 0
Burst Time of P1: 2

Arrival Time of P2: 1
Burst Time of P2: 2

Arrival Time of P3: 5
Burst Time of P3: 3

Arrival Time of P4: 6
Burst Time of P4: 4

Process AT    BT    CT    TAT    WT
1      0      2      2      2      0
2      1      2      4      3      1
3      5      3      8      3      0
4      6      4     12      6      2
Average TAT: 3.50

Average WT: 0.75
```

Program 1 b)

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

a) Priority (non pre-emptive)

// 2. Pri non pre

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid, at, bt, priority, ct, wt, tat, is_completed;
```

```
};
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    struct Process p[n];
```

```
    for(int i=0; i<n; i++) {
```

```
        p[i].pid = i+1;
```

```
        printf("Enter AT, BT and Priority for P%d: ", i+1);
```

```
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
```

```
        p[i].is_completed = 0;
```

```
    }
```

```

int completed = 0, time = 0, min_pri;
int selected = -1;

while(completed != n) {
    min_pri = 10000;
    selected = -1;

    for(int i=0; i<n; i++) {
        if((p[i].at <= time) && (p[i].is_completed == 0) && (p[i].priority < min_pri)) {
            min_pri = p[i].priority;
            selected = i;
        }
    }

    if(selected == -1) {
        time++;
        continue;
    }

    time += p[selected].bt;
    p[selected].ct = time;
    p[selected].tat = p[selected].ct - p[selected].at;
    p[selected].wt = p[selected].tat - p[selected].bt;
    p[selected].is_completed = 1;
    completed++;
}

printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");

```

```

for(int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].priority, p[i].ct, p[i].wt, p[i].tat);
}
int sumtat=0;
int sumwt=0;
for(int i=0;i<n;i++){
    sumtat+=p[i].tat
    ;
} sumwt+=p[i].wt;
printf("\n      avg      tat      %.2f",
(float)sumtat/(float)n); printf("\n  avg  tat
%.2f", (float)sumwt/(float)n);
printf("\n1BM24CS028");
return 0;
}

```

b) Priority (pre-emptive)

```

#include <stdio.h>
struct Process{
    int id, at, bt, pri, ct, wt, tat,rt;
};
int main(){
    int n;
    printf("\nenter no of process: ");
    scanf("%d",&n);
    struct Process p[n];
    for(int i=0;i<n;i++){
        p[i].id=i+1;
        printf("enter at bt and priority of P%d
",p[i].id);          scanf("%d      %d
%d",&p[i].at,&p[i].bt,&p[i].pri); p[i].rt=p[i].bt;

    }
    int currt=0;
    int completed=0;
    int pri_max;

```

```

int idx;
while(completed<n){
    pri_max=1000;
    idx=-1;
    for(int i=0;i<n;i++){
        if((p[i].at<=currt)&&(p[i].rt>0)){
            if(p[i].pri<pri_max){
                pri_max=p[i].pri;
                idx=i;
            }
        }
    }
    if(idx==-1){
        currt++;
    }else{
        p[idx].rt=p[idx].rt-1;
        currt++;
        if(p[idx].rt==0){
            p[idx].ct=currt;
            p[idx].tat=p[idx].ct-p[idx].at;
            p[idx].wt=p[idx].tat-p[idx].bt;
            completed++;
        }
    }
}
printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");
for(int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].at, p[i].bt, p[i].pri, p[i].ct, p[i].wt, p[i].tat);
}
int sumtat=0;
int sumwt=0;
for(int i=0;i<n;i++){
    sumtat+=p[i].tat
    ;
} sumwt+=p[i].wt;
printf("\n      avg      tat      %.2f",
(float)sumtat/(float)n); printf("\n avg tat
%.2f", (float)sumwt/(float)n);
printf("\n1BM24CS028");
return 0;
}

```


c) Round Robin

```
#include <stdio.h>

struct Process {
    int pid, at, bt, ct, tat, wt, rt;
};

int main() {
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Time Quantum: ");
    scanf("%d", &tq);

    struct Process p[n];
    int rem_bt[n]; // remaining burst times
    int readyQ[100]; // simple queue for process indices
    int front = 0, rear = 0;

    printf("Enter arrival times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &p[i].at);
        p[i].pid = i + 1;
    }

    printf("Enter burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &p[i].bt);
        rem_bt[i] = p[i].bt;
        p[i].rt = p[i].bt; // here rt is remaining time
        p[i].ct = p[i].tat = p[i].wt = 0;
    }

    int time = 0, completed = 0;
    int visited[n];
    for (int i = 0; i < n; i++) visited[i] = 0;

    // initially push processes that arrive at time 0
    for (int i = 0; i < n; i++) {
        if (p[i].at == 0) {
```

```

    readyQ[rear++] = i;
    visited[i] = 1;
}
}

while (completed < n) {
    if (front == rear) { // queue empty → advance time
        time++;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && p[i].at <= time) {
                readyQ[rear++] = i;
                visited[i] = 1;
            }
        }
        continue;
    }

    int idx = readyQ[front++]; // dequeue

    if (rem_bt[idx] > tq) {
        time += tq;
        rem_bt[idx] -= tq;
        p[idx].rt = rem_bt[idx];
    } else {
        time += rem_bt[idx];
        rem_bt[idx] = 0;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        p[idx].rt = 0;
        completed++;
    }

    // add newly arrived processes to queue
    for (int i = 0; i < n; i++) {
        if (!visited[i] && p[i].at <= time) {
            readyQ[rear++] = i;
            visited[i] = 1;
        }
    }

    // if current process still has burst time left, push it back

```

```

        if (rem_bt[idx] > 0) {
            readyQ[rear++] = idx;
        }
    }

    printf("\nRound Robin Scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].ct,
            p[i].tat, p[i].wt, p[i].rt);
    }

    // Calculate averages
    float avg_TAT = 0, avg_WT = 0;
    for (int i = 0; i < n; i++) {
        avg_TAT += p[i].tat;
        avg_WT += p[i].wt;
    }

    printf("\nAverage turnaround time: %.2f ms\n", avg_TAT / n);
    printf("Average waiting time: %.2f ms\n", avg_WT / n);
    printf("\n1BM24CS028\n");

    return 0;
}

```

Result:

a) Priority non pre-emptive

```
C:\Users\BMSCECSE-L4\Desktop X + v
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0 3 5
Enter AT, BT and Priority for P2: 2 2 3
Enter AT, BT and Priority for P3: 3 5 2
Enter AT, BT and Priority for P4: 4 4 4
Enter AT, BT and Priority for P5: 6 1 1

PID    AT    BT    Pri    CT    WT    TAT
1       0     3     5      3     0     3
2       2     2     3     11     7     9
3       3     5     2      8     0     5
4       4     4     4     15     7    11
5       6     1     1      9     2     3

Process returned 0 (0x0)   execution time : 94.456 s
Press any key to continue.
```

b) Priority pre-emptive

```
C:\Users\BMSCECSE-L4\Desktop X + v
enter no of process: 5
enter at bt and priority of P1 0 3 5
enter at bt and priority of P2 2 2 3
enter at bt and priority of P3 3 5 2
enter at bt and priority of P4 4 4 4
enter at bt and priority of P5 6 1 1

PID    AT    BT    Pri    CT    WT    TAT
1       0     3     5     15    12    15
2       2     2     3     10     6     8
3       3     5     2      9     1     6
4       4     4     4     14     6    10
5       6     1     1      7     0     1

avg tat 8.00
avg tat 5.00
```

c) Round Robin

```

Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
5 3 1 2 3

Round Robin Scheduling:

```

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	8	0
2	1	3	12	11	8	0
3	2	1	5	3	2	0
4	3	2	9	6	4	0
5	4	3	14	10	7	0

```

Average turnaround time: 8.60 ms
Average waiting time: 5.80 ms

Process returned 0 (0x0)   execution time : 36.815 s
Press any key to continue.

```

Program -2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

a) using FCFS

```
#include <stdio.h>

int main() { int bt[20], wt[20], tat[20], ct[20],
queue[20]; int n; float wtavg = 0, tatavg = 0;
int
sys_bt[20], user_bt[20];
int sys_count = 0, user_count = 0;

printf("Enter the number of processes: "); scanf("%d", &n);

for (int i = 0; i < n; i++) { printf("Enter Burst Time for Process %d: ", i + 1);
scanf("%d", &bt[i]); printf("Enter Queue Number (1 = System, 2 = User) for
Process %d: ", i + 1);

scanf("%d", &queue[i]);

if (queue[i] == 1) { sys_bt[sys_count++]
= bt[i];
} else if (queue[i] == 2) { user_bt[user_count++]
= bt[i];
} } }
```

```
total_count = 0; int current_time  
= 0;
```

```
for (int i = 0; i < sys_count; i++) { if  
    (total_count == 0) { wt[total_count]  
        = 0;  
    } else {      wt[total_count] = current_time;  
    }  
}
```

```
tat[total_count] = wt[total_count] + sys_bt[i]; ct[total_count]  
= current_time + sys_bt[i];
```

```
current_time = ct[total_count];
```

```
wtavg += wt[total_count]; tatavg  
+= tat[total_count]; total_count++;  
}
```

```
for (int i = 0; i < user_count; i++) { if  
    (total_count == 0) { wt[total_count]  
        = 0;  
    } else {      wt[total_count] = current_time;  
    }  
}
```

```
tat[total_count] = wt[total_count] + user_bt[i]; ct[total_count] = current_time +  
user_bt[i];
```

```
current_time      =
```

```

        ct[total_count]; wtavg += wt[total_count];
        tatavg +=
        tat[total_count]; total_count++;
    }

    printf("\nPROCESS\tBURST    TIME\tQUEUE\tCOMPLETION    TIME\tWAITING
    TIME\tTURNAROUND TIME\n"); int sys_index = 0, user_index = 0; for (int i = 0; i <
    total_count; i++) { if (sys_index < sys_count) { printf("%d\t%d\t\tSystem\t%d\t\t%d\t\t%d\n", i
    + 1, sys_bt[sys_index], ct[i], wt[i], tat[i]); sys_index++;
        } else if (user_index < user_count) { printf("%d\t%d\t\tUser\t%d\t\t%d\t\t%d\n", i + 1,
        user_bt[user_index], ct[i], wt[i], tat[i]); user_index++;
        }
    }
}

printf("\nAverage Waiting Time: %.2f", wtavg / total_count); printf("\nAverage Turnaround
Time: %.2f\n", tatavg / total_count);

return 0;
}
#include <stdio.h> void roundRobin(int bt[],
    using Round_robin
    sys_count, int user_count)
    int n, int quantum, int queue[], int

{ int remaining_bt[20]; int wt[20] = {0},
    tat[20] = {0}, ct[20] = {0}; int total_count =
    sys_count + user_count; int queue_index
    = 0; int current_time = 0,

    total_bt = 0; float wtavg = 0, tatavg = 0;

```



```

for (int i = 0; i < total_count; i++) { remaining_bt[i]
    = bt[i]; total_bt
    += bt[i];
}

```

```

while (total_bt > 0) { for (int i = 0; i <
    total_count; i++) { if (remaining_bt[i] > 0) { int time_slice = (remaining_bt[i]
        <= quantum) ? remaining_bt[i] : quantum; current_time += time_slice;
        remaining_bt[i] -= time_slice; total_bt -= time_slice;

        if (remaining_bt[i] == 0) { ct[i]
            = current_time; tat[i] = ct[i] -
            (total_bt - bt[i]); wt[i] = tat[i] -
            bt[i];
        }
    }
}
}

```

```

for (int i = 0; i < total_count; i++) {
    wtavg += wt[i]; tatavg
    += tat[i];
}

```

```

printf("\nPROCESS\tBURST TIME\tQUEUE\tCOMPLETION TIME\tWAITING
TIME\tTURNAROUND TIME\n"); for (int i = 0; i < total_count; i++) { if
(queue[i] == 1) {
    printf("%d\t%d\t\tSystem\t%d\t\t%d\t\t%d\n", i + 1, bt[i], ct[i], wt[i], tat[i]);
}
}

```

```

        } else if (queue[i] == 2) { printf("%d\t%d\t\tUser\t%d\t\t%d\t\t%d\n", i + 1, bt[i], ct[i],
            wt[i], tat[i]); }
    }

    printf("\nAverage Waiting Time: %.2f", wtavg / total_count); printf("\nAverage Turnaround
    Time: %.2f\n", tatavg / total_count);
}

int main() { int bt[20], queue[20];
    int    n, quantum; int
    sys_count = 0, user_count = 0;

    printf("Enter the number of processes: "); scanf("%d", &n);

    for (int i = 0; i < n; i++) { printf("Enter Burst Time
        for Process %d: ", i + 1); scanf("%d", &bt[i]); printf("Enter Queue Number (1 =
        System, 2 = User) for Process %d: ", i + 1);
        scanf("%d", &queue[i]);
        if (queue[i] == 1) {
            sys_count++;
        } else if (queue[i] == 2) {
            user_count++;
        }
    }

    printf("Enter the Time Quantum: "); scanf("%d",
    &quantum);

```

```

roundRobin(bt, n, quantum, queue, sys_count, user_count); return
0;
}

```

Result:

```

Enter the number of processes: 5
Enter Burst Time for Process 1: 4
Enter Queue Number (1 = System, 2 = User) for Process 1: 2
Enter Burst Time for Process 2: 7
Enter Queue Number (1 = System, 2 = User) for Process 2: 1
Enter Burst Time for Process 3: 8
Enter Queue Number (1 = System, 2 = User) for Process 3: 1
Enter Burst Time for Process 4: 1
Enter Queue Number (1 = System, 2 = User) for Process 4: 2
Enter Burst Time for Process 5: 4
Enter Queue Number (1 = System, 2 = User) for Process 5: 1

```

PROCESS	BURST TIME	QUEUE	COMPLETION TIME	WAITING TIME	TURNAROUND TIME
1	7	System	7	0	7
2	8	System	15	7	15
3	4	System	19	15	19
4	4	User	23	19	23
5	1	User	24	23	24

Average Waiting Time: 12.80
Average Turnaround Time: 17.60

```

Enter the number of processes: 5
Enter Burst Time for Process 1: 4
Enter Queue Number (1 = System, 2 = User) for Process 1: 2
Enter Burst Time for Process 2: 7
Enter Queue Number (1 = System, 2 = User) for Process 2: 1
Enter Burst Time for Process 3: 8
Enter Queue Number (1 = System, 2 = User) for Process 3: 1
Enter Burst Time for Process 4: 1
Enter Queue Number (1 = System, 2 = User) for Process 4: 2
Enter Burst Time for Process 5: 4
Enter Queue Number (1 = System, 2 = User) for Process 5: 1
Enter the Time Quantum: 3

```

PROCESS	BURST TIME	QUEUE	COMPLETION TIME	WAITING TIME	TURNAROUND TIME
1	4	User	14	4	8
2	7	System	22	20	27
3	8	System	24	24	32
4	1	User	10	-4	-3
5	4	System	21	18	22

Average Waiting Time: 12.40
Average Turnaround Time: 17.20

Program -3

Question:

Write a Cprogram to simulate Real-Time CPU Scheduling algorithms

a) Rate- Monotonic

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include
<stdbool.h>
#define MAX_TASKS 10

typedef struct {
    int    id;    int
    execution_time; int
    period;    int
    time_remaining; int
    next_start_time;
} Task;

// Function to calculate GCD int gcd(int
a, int b) {
    return (b == 0) ? a : gcd(b, a % b);
}

// Function to calculate LCM of all task periods
int find_lcm(int periods[], int n) { int lcm =
periods[0]; for (int i = 1; i < n; i++) {
```

```

    lcm = (lcm * periods[i]) / gcd(lcm, periods[i]);
}
return lcm;}

```

```

void rate_monotonic(Task tasks[], int n) {

```

```

    int periods[MAX_TASKS];
    for (int i = 0; i < n; i++) { periods[i]
        = tasks[i].period;
    }

```

```

    int simulation_time = find_lcm(periods, n); // Set simulation time to LCM of periods
    printf("\nRate-Monotonic Scheduling (Simulating till time = %d):\n", simulation_time);

```

```

    for (int time = 0; time < simulation_time; time++) { int chosen_task
        = -1;

```

```

        // Check if any task arrives at this
        time for (int i = 0; i < n; i++) { if (time
            == tasks[i].next_start_time) {
                tasks[i].time_remaining =
                tasks[i].execution_time; tasks[i].next_start_time += tasks[i].period;
            }
        }
    }

```

```

    // Pick the highest-priority (shortest period) ready task for (int i = 0; i
    < n; i++) { if (tasks[i].time_remaining > 0) { if (chosen_task == -1 ||
    tasks[i].period < tasks[chosen_task].period) { chosen_task = i;
        }
    }

```

```

    }
}

// Execute the chosen task or idle if (chosen_task != -1)
{   printf("Time    %d:    Task    %d\n",    time,
tasks[chosen_task].id);
tasks[chosen_task].time_remaining--;
} else { printf("Time %d: Idle\n", time);
} }
}

```

```

int main() {
    int n;   printf("Enter the number of tasks: ");
    scanf("%d",
    &n);

    Task tasks[MAX_TASKS];
    for (int i = 0; i < n; i++) { printf("Enter execution time and
    period for Task %d: ", i + 1); scanf("%d %d",
    &tasks[i].execution_time, &tasks[i].period); tasks[i].id =
        i        +        1;        tasks[i].time_remaining =
        0; tasks[i].next_start_time = 0;
    }
}

```

```

    rate_monotonic(tasks, n); return 0;
}

```

b) earliest deadline first

```
#include <stdio.h>
```

```
struct Task {
```

```

int pid;
int burst;
int deadline;
int period;
int rem;
int abs_dl;
};

```

```

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

```

```

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

```

```

int main() {
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct Task tks[n];
    int hyperperiod = 1;

    for (int i = 0; i < n; i++) {
        printf("Enter PID, Burst, Deadline, and Period for process %d: ", i + 1);
        scanf("%d %d %d %d", &tks[i].pid, &tks[i].burst, &tks[i].deadline, &tks[i].period);
    }
}

```

```

    tks[i].rem = 0;
    tks[i].abs_dl = 0;
    hyperperiod = lcm(hyperperiod, tks[i].period);
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", tks[i].pid, tks[i].burst, tks[i].deadline, tks[i].period);
}

printf("Scheduling occurs for %d ms\n\n", hyperperiod);

int prev_selected = -2; // Track previous state to only print context switches

for (int time = 0; time < hyperperiod; time++) {
    for (int i = 0; i < n; i++) {
        if (time % tks[i].period == 0) {
            tks[i].rem = tks[i].burst;
            tks[i].abs_dl = time + tks[i].deadline;
        }
    }
}

int min_dl = 999999;
int selected = -1;

for (int i = 0; i < n; i++) {
    if (tks[i].rem > 0 && tks[i].abs_dl < min_dl) {
        min_dl = tks[i].abs_dl;
        selected = i;
    }
}

```



```

    }
}
if (selected != prev_selected) {
    if (selected != -1) {
        printf("%dms onwards: Process %d running\n", time, tks[selected].pid);
    } else {
        printf("%dms onwards: CPU is idle\n", time);
    }
    prev_selected = selected;
}
if (selected != -1) {
    tks[selected].rem--;
}
}
printf("\n1BM24CS028");

return 0;
}

```

Result:

```

Enter the number of tasks: 2
Enter execution time and period for Task 1: 4
8
Enter execution time and period for Task 2: 1
4
Rate-Monotonic Scheduling (Simulating till time = 8):
Time 0: Task 2
Time 1: Task 1
Time 2: Task 1
Time 3: Task 1
Time 4: Task 2
Time 5: Task 1
Time 6: Idle
Time 7: Idle

```

Program 3.2

Earliest Deadline First

```
#include <stdio.h>

struct Task {
    int pid;
    int burst;
    int deadline;
    int period;
    int rem;
    int abs_dl;
};

int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    struct Task tks[n];
```

```

int hyperperiod = 1;

for (int i = 0; i < n; i++) {
    printf("Enter PID, Burst, Deadline, and Period for process %d: ", i + 1);
    scanf("%d %d %d %d", &tkes[i].pid, &tkes[i].burst, &tkes[i].deadline, &tkes[i].period);
    tkes[i].rem = 0;
    tkes[i].abs_dl = 0;
    hyperperiod = lcm(hyperperiod, tkes[i].period);
}

printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", tkes[i].pid, tkes[i].burst, tkes[i].deadline, tkes[i].period);
}

printf("Scheduling occurs for %d ms\n\n", hyperperiod);

int prev_selected = -2; // Track previous state to only print context switches

for (int time = 0; time < hyperperiod; time++) {
    for (int i = 0; i < n; i++) {
        if (time % tkes[i].period == 0) {
            tkes[i].rem = tkes[i].burst;
            tkes[i].abs_dl = time + tkes[i].deadline;
        }
    }
}

int min_dl = 999999;

int selected = -1;

```

```

for (int i = 0; i < n; i++) {
    if (tks[i].rem > 0 && tks[i].abs_dl < min_dl) {
        min_dl = tks[i].abs_dl;
        selected = i;
    }
}

// Print only if there is a context switch or CPU state change
if (selected != prev_selected) {
    if (selected != -1) {
        printf("%dms onwards: Process %d running\n", time, tks[selected].pid);
    } else {
        printf("%dms onwards: CPU is idle\n", time);
    }
    prev_selected = selected;
}
if (selected != -1) {
    tks[selected].rem--;
}
}
printf("\n1BM24CS028");

return 0;
}

```

Result:

```

PS C:\Users\BMSCECSE-L4\Desktop\OS-scheduling> .\edf.exe
Enter the number of processes: 3
Enter PID, Burst, Deadline, and Period for process 1: 2 2 4 5
Enter PID, Burst, Deadline, and Period for process 2: 1 3 7 20
Enter PID, Burst, Deadline, and Period for process 3: 3 2 8 10

PID    Burst  Deadline  Period
2       2      4         5
1       3      7        20
3       2      8        10
Scheduling occurs for 20 ms

0ms onwards: Process 2 running
2ms onwards: Process 1 running
5ms onwards: Process 3 running
7ms onwards: Process 2 running
9ms onwards: CPU is idle
10ms onwards: Process 2 running
12ms onwards: Process 3 running
14ms onwards: CPU is idle
15ms onwards: Process 2 running
17ms onwards: CPU is idle

PS C:\Users\BMSCECSE-L4\Desktop\OS-scheduling>

```

propositional share scheduling

```
#include<stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
typedef struct
```

```
{
```

```
char name[5]; int tickets; }
```

```
Process;
```

```
int main() {
```

```
int n, total_tickets = 0; float total_T
```

```
=0.0; printf("Enter the number of
```

```
Processes: "); scanf("%d", &n);
```

```
Process p[n];
```

```
srand(time(NULL));
```

```
for (int i = 0; i < n; i++)
```

```
{ printf("\nProcess %d:\n", i + 1);
```

```
sprintf(p[i].name, "P%d", i + 1);
```

```

printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;

scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets   picked:   %d,   Winner:   %s\n",
winning_ticket, p[winner_index].name);
}
for (int i = 0; i < n; i++)
{

```

```
printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,  
((p[i].tickets / total_T) * 100));
```

```
}return 0;}  
Result.
```

```
Enter the number of Processes: 3  
  
Process 1:  
Tickets: 10  
  
Process 2:  
Tickets: 20  
  
Process 3:  
Tickets: 30  
  
--- Proportional Share Scheduling ---  
Enter the Time Period for scheduling: 5  
Tickets picked: 38, Winner: P3  
Tickets picked: 3, Winner: P1  
Tickets picked: 3, Winner: P1  
Tickets picked: 48, Winner: P3  
Tickets picked: 36, Winner: P3  
  
The Process: P1 gets 16.67% of Processor Time.  
  
The Process: P2 gets 33.33% of Processor Time.  
  
The Process: P3 gets 50.00% of Processor Time.  
PS C:\Users\BMSCECSE-L4\Desktop\LAB_4> █
```

Program -4

Question:

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code:

a) Producer-Consumer problem using semaphores.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#define MAX_ITEMS 5
```

```
#define BUFFER_SIZE 5
```

```
int
```

```
buffer[BUFFER_SIZE];
```

```
int in = 0, out = 0; sem_t
```

```
mutex; sem_t full; sem_t
```

```
empty;
```

```
int produced_count = 0, consumed_count = 0;
```

```
void *producer(void *arg) {
```

```
    sem_wait(&empty)
```

```
    ;
```

```
    sem_wait(&mutex)
```

```
    buffer[in] = produced_count + 1;
```



```

printf("Producer has produced: Item %d\n", buffer[in]); in
= (in + 1) % BUFFER_SIZE;
produced_count++;
sem_post(&mutex); sem_post(&full);
pthread_exit(NULL);
}

void *consumer(void *arg) {
    sem_wait(&full); sem_wait(&mutex);

    int last_item_index = (in - 1 + BUFFER_SIZE) %
    BUFFER_SIZE; printf("Consumer has consumed: Item %d\n",
    buffer[last_item_index]); buffer[last_item_index] = 0;
    consumed_count++;

    in = (in - 1 + BUFFER_SIZE) % BUFFER_SIZE;

    sem_post(&mutex); sem_post(&empty);
    pthread_exit(NULL);
}

int main() { pthread_t prod_thread,
    cons_thread; int choice;

    sem_init(&mutex, 0, 1);
    sem_init(&full, 0, 0);
    sem_init(&empty, 0,
    MAX_ITEMS); while (1) { printf("Enter
        1.Producer 2.Consumer

```

```

3.exit\n"); printf("Enter choice: "); scanf("%d",
&choice); switch (choice) { case
1:
    if (produced_count < MAX_ITEMS) {
        pthread_create(&prod_thread, NULL, producer,
        NULL); pthread_join(prod_thread, NULL);
    } else { printf("Buffer is full. Cannot produce more items.\n");
    } break;
case 2:
    if (consumed_count < produced_count) {
        pthread_create(&cons_thread, NULL, consumer, NULL); pthread_join(cons_thread,
        NULL);
    } else { printf("Buffer is empty. Cannot consume more
        items.\n");
    } break;
case 3:
    sem_destroy(&mutex);
    sem_destroy(&full);
    sem_destroy(&empty); return
    0;
default: printf("Invalid
        choice.\n");
    }
}
return 0;
}

```

b) Dining-Philosopher's problem

```
#include <stdio.h>
```

```

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>


#define NUM_PHILOSOPHERS 5

#define THINKING 0

#define HUNGRY 1

#define EATING 2


int state[NUM_PHILOSOPHERS];
int phil_ids[NUM_PHILOSOPHERS];


sem_t mutex;
sem_t S[NUM_PHILOSOPHERS];


void test(int i) {
    if (state[i] == HUNGRY && state[(i + 4) %
        NUM_PHILOSOPHERS] != EATING && state[(i
        + 1) % NUM_PHILOSOPHERS] != EATING) {

        state[i] = EATING; sleep(1); printf("Philosopher %d takes forks
        %d and %d and starts eating\n", i + 1, (i + 4) %
        NUM_PHILOSOPHERS + 1, i + 1); sem_post(&S[i]);
    }
}


void take_fork(int i) { sem_wait(&mutex);

```

```

state[i]      =
HUNGRY;
printf("Philosopher %d is hungry\n", i + 1);
test(i); sem_post(&mutex); sem_wait(&S[i]);
sleep(1);
}

void put_fork(int i) { sem_wait(&mutex); state[i] = THINKING; printf("Philosopher
%d puts down forks %d and %d and starts thinking\n", i + 1, (i + 4) %
NUM_PHILOSOPHERS + 1, i + 1); test((i + 4) % NUM_PHILOSOPHERS); test((i +
1)
% NUM_PHILOSOPHERS); sem_post(&mutex);
}

void* philosopher(void* num) { int i
    = *(int*)num;

    while (1)    { printf("Philosopher  %d is
        thinking\n", i + 1); sleep(1); take_fork(i); sleep(2);
        put_fork(i);
    }

    return NULL;
}

int main() { int i;
    pthread_t
    thread_id[NUM_PHILOSOPHERS]; sem_init(&mutex,
    0, 1); for (i = 0; i <

```

```

NUM_PHILOSOPHERS; i++) {
    sem_init(&S[i], 0, 0);
    phil_ids[i] = i;
}

for (i = 0; i < NUM_PHILOSOPHERS; i++) {    pthread_create(&thread_id[i],
        NULL, philosopher,  &phil_ids[i]);
    printf("Philosopher %d is seated at the table\n", i + 1);
}

for (i = 0; i < NUM_PHILOSOPHERS; i++) {
    pthread_join(thread_id[i], NULL);
}

return 0;
}

```

Result:

```

Enter 1.Producer 2.Consumer 3.exit
Enter choice: 1
Producer has produced: Item 1
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 1
Producer has produced: Item 2
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 1
Producer has produced: Item 3
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 1
Producer has produced: Item 4
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 2
Consumer has consumed: Item 4
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 2
Consumer has consumed: Item 3
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 2
Consumer has consumed: Item 2
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 2
Consumer has consumed: Item 1
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 2
Buffer is empty. Cannot consume more items.
Enter 1.Producer 2.Consumer 3.exit
Enter choice: 3

```

```

Philosopher 1 is seated at the table
Philosopher 2 is seated at the table
Philosopher 3 is seated at the table
Philosopher 4 is seated at the table
Philosopher 5 is seated at the table
Philosopher 1 is thinking
Philosopher 2 is thinking
Philosopher 3 is thinking
Philosopher 4 is thinking
Philosopher 5 is thinking
Philosopher 3 is hungry
Philosopher 3 takes forks 2 and 3 and starts eating
Philosopher 1 is hungry
Philosopher 1 takes forks 5 and 1 and starts eating
Philosopher 2 is hungry
Philosopher 5 is hungry
Philosopher 4 is hungry
Philosopher 3 puts down forks 2 and 3 and starts thinking
Philosopher 4 takes forks 3 and 4 and starts eating
Philosopher 3 is thinking
Philosopher 1 puts down forks 5 and 1 and starts thinking
Philosopher 2 takes forks 1 and 2 and starts eating
Philosopher 3 is hungry
Philosopher 1 is thinking
Philosopher 1 is hungry
Philosopher 4 puts down forks 3 and 4 and starts thinking
Philosopher 5 takes forks 4 and 5 and starts eating
Philosopher 4 is thinking
Philosopher 2 puts down forks 1 and 2 and starts thinking
Philosopher 3 takes forks 2 and 3 and starts eating
Philosopher 2 is thinking
Philosopher 4 is hungry
Philosopher 2 is hungry
Philosopher 5 puts down forks 4 and 5 and starts thinking
Philosopher 1 takes forks 5 and 1 and starts eating
Philosopher 5 is thinking
Philosopher 3 puts down forks 2 and 3 and starts thinking
Philosopher 4 takes forks 3 and 4 and starts eating

```

Program -5

Question:

Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

Code:

```
#include<stdio.h>

> int main() { int n, m, i, j, k; printf("Enter the number
of processes: "); scanf("%d",
&n); printf("Enter the number of
resources: "); scanf("%d", &m); int
allocation[n][m]; printf("Enter the Allocation Matrix:\n"); for (i = 0; i <
n; i++){

    for (j = 0; j < m; j++){
        scanf("%d", &allocation[i][j]);
    }
} int max[n][m]; printf("Enter the
MAX Matrix:\n"); for (i = 0; i < n;
i++){

    for (j = 0; j < m; j++){
        scanf("%d", &max[i][j]);
    } } int available[m];

printf("Enter the Available Resources:\n"); for
(i = 0; i < m; i++){

    scanf("%d", &available[i]);
} int f[n], ans[n], ind =
0; for (k = 0; k < n;
k++){ f[k] = 0; } int
need[n][m]; for (i =
```

```

0; i < n; i++){ for (j = 0; j < m; j++){
    need[i][j] = max[i][j] - allocation[i][j];
} } int y = 0; for (k =
0; k < n; k++){ for (i =
0; i < n; i++){ if (f[i] == 0){
    int flag = 0; for (j = 0; j
    < m; j++){
        if (need[i][j] > available[j]){
            flag = 1; break;
        } } if (flag
        == 0){ ans[ind++] = i; for
        (y =
        0; y < m; y++){ available[y] +=
        allocation[i][y];
        } f[i] =
        1; }

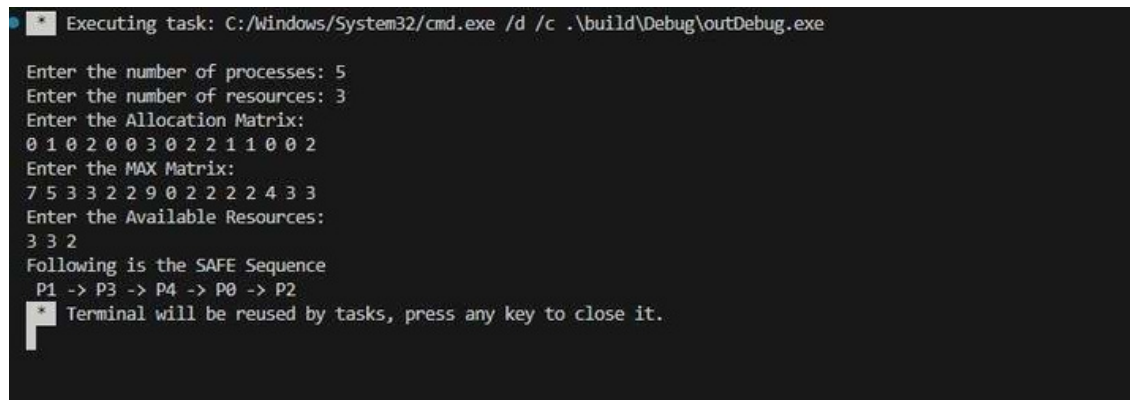
    }
} } int flag = 1; for
(i = 0; i < n; i++){ if
(f[i] == 0){
    flag = 0;    printf("The following system is not
    safe\n"); break; } } if (flag
    == 1){    printf("Following is the SAFE
    Sequence\n"); for
    (i = 0; i < n - 1; i++){ printf("
    P%d ->", ans[i]);

```



```
    } printf(" P%d\n", ans[n -  
    1]);  
} return  
0;  
}
```

Result:



```
Executing task: C:/Windows/System32/cmd.exe /d /c .\build\Debug\outDebug.exe  
  
Enter the number of processes: 5  
Enter the number of resources: 3  
Enter the Allocation Matrix:  
0 1 0 2 0 0 3 0 2 2 1 1 0 0 2  
Enter the MAX Matrix:  
7 5 3 3 2 2 9 0 2 2 2 2 4 3 3  
Enter the Available Resources:  
3 3 2  
Following is the SAFE Sequence  
P1 -> P3 -> P4 -> P0 -> P2  
Terminal will be reused by tasks, press any key to close it.
```

Program -6

Question:

Write a C program to simulate the following contiguous memory allocation techniques. a)

Worst-fit

b) Best-fit

c) First-fit

Code:

```
#include <stdio.h>
```

```
#define MAX 10
```

```
void firstFit(int blockSize[], int blocks, int processSize[], int processes) {
```

```
    int allocation[MAX]; for (int i = 0; i < processes;
```

```
    i++) allocation[i] = -1;
```

```
    for (int i = 0; i < processes; i++) { for (int
```

```
        j = 0; j < blocks; j++) { if (blockSize[j]
```

```
        >= processSize[i]) { allocation[i] = j;
```

```
        blockSize[j] -= processSize[i]; break;
```

```
        }
```

```
    }
```

```
}
```

```
printf("\nFirst-Fit Allocation:\n"); for (int i = 0; i < processes; i++) { if (allocation[i] !=
```

```
-1) printf("Process %d of size %d -> Block %d\n", i + 1, processSize[i], allocation[i] +
```

```
1); else printf("Process %d of size %d -> Not Allocated\n", i + 1, processSize[i]);
```

```
}}
```

```

void bestFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[MAX];
    for (int i = 0; i < processes; i++) allocation[i] = -1;

    for (int i = 0; i < processes; i++) { int best = -1; for (int j = 0;
        j < blocks; j++) { if (blockSize[j] >= processSize[i]) { if
        (best == -1 || blockSize[j] <
        blockSize[best]) best = j; } } if (best
        != -1) { allocation[i] = best;
        blockSize[best] -= processSize[i];
        }
    }
}

printf("\nBest-Fit Allocation:\n"); for (int i = 0; i < processes; i++) { if (allocation[i] !=
-1) printf("Process %d of size %d -> Block %d\n", i + 1, processSize[i], allocation[i] +
1); else printf("Process %d of size %d -> Not Allocated\n", i + 1, processSize[i]);
}
}

```

```

void worstFit(int blockSize[], int blocks, int processSize[], int processes) {
    int allocation[MAX]; for (int i = 0; i < processes;
    i++) allocation[i] = 1; for (int i = 0; i < processes;
    i++) { int worst = -
    1; for (int j = 0; j < blocks; j++) { if (blockSize[j] >=
    processSize[i])
    { if (worst == -1 || blockSize[j] > blockSize[worst]) worst = j; }
    }
    if (worst != -1) { allocation[i] =
    worst; blockSize[worst] -= processSize[i]; }
}

```

```

    }

    printf("\nWorst-Fit Allocation:\n"); for (int i = 0; i < processes; i++) { if (allocation[i]
    != - 1) printf("Process %d of size %d -> Block %d\n", i + 1, processSize[i],
    allocation[i]); else printf("Process %d of size %d -> Not Allocated\n", i + 1, processSize[i]);
    }
}

```

```

int main() {    int blockSize[MAX], processSize[MAX], blocks,
    processes, choice;

```

```

    printf("Enter number of memory blocks:
    "); scanf("%d", &blocks); printf("Enter
    size of each block:\n"); for (int i = 0; i <
    blocks; i++) { printf("Block %d: ", i + 1);
    scanf("%d",

```

```

        &blockSize[i]);
    } printf("Enter number of processes:
    ");

```

```

    scanf("%d", &processes);
    printf("Enter size of each process:\n");
    for (int i = 0; i < processes; i++) {
    printf("Process %d: ", i + 1);
    scanf("%d", &processSize[i]);
    }

```

```

printf("\nMemory Allocation Techniques:\n"); printf("1.
First Fit\n2. Best Fit\n3. Worst Fit\nEnter choice: ");
scanf("%d", &choice);

int originalBlockSize[MAX];
for (int i = 0; i < blocks; i++) originalBlockSize[i] = blockSize[i];

switch (choice) { case 1: firstFit(originalBlockSize, blocks,
    processSize, processes); break;
    case 2: for (int i = 0; i < blocks; i++) blockSize[i] =
    originalBlockSize[i]; bestFit(blockSize, blocks, processSize,
    processes); break;
    case 3:
        for (int i = 0; i < blocks; i++) blockSize[i] = originalBlockSize[i]; worstFit(blockSize, blocks,
        processSize, processes);
        break;
    default: printf("Invalid
        choice.\n");
}

return 0;}

```

Result:

```

Enter number of memory blocks: 5
Enter size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter number of processes: 4
Enter size of each process:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426

Memory Allocation Techniques:
1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 1

First-Fit Allocation:
Process 1 of size 212 -> Block 2
Process 2 of size 417 -> Block 5
Process 3 of size 112 -> Block 2
Process 4 of size 426 -> Not Allocated

```

```

Enter number of memory blocks: 5
Enter size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter number of processes: 4
Enter size of each process:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426

Memory Allocation Techniques:
1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 2

Best-Fit Allocation:
Process 1 of size 212 -> Block 4
Process 2 of size 417 -> Block 2
Process 3 of size 112 -> Block 3
Process 4 of size 426 -> Block 5

```

```

Enter number of memory blocks: 5
Enter size of each block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600
Enter number of processes: 4
Enter size of each process:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426

Memory Allocation Techniques:
1. First Fit
2. Best Fit
3. Worst Fit
Enter choice: 3

Worst-Fit Allocation:
Process 1 of size 212 -> Block 5
Process 2 of size 417 -> Block 2
Process 3 of size 112 -> Block 5
Process 4 of size 426 -> Not Allocated

```

Program -7

Question:

Write a Cprogram to simulate page replacement algorithms.

- a) FIFO
- b) LRU
- c) Optimal

Code:

```
#include <stdio.h>
#include <limits.h>

void fifo(int pages[], int n, int capacity) {
    int frames[capacity], index = 0, faults = 0; for
    (int i = 0; i < capacity; i++) frames[i] = -1;

    printf("\nFIFO Page Replacement\n");
    for (int i = 0; i < n; i++) { int found = 0;
    for (int j = 0; j < capacity; j++) {
        if (frames[j] == pages[i]) {
            found    =    1;
            break;
        } } if
    (!found) {
        frames[index]    =
            pages[i]; index = (index + 1)
            % capacity; faults++;
    }
    printf("Frames: "); for (int j = 0;
    j < capacity; j++) {
```

```

        if (frames[j] == -1) printf("
            - ");
        else
            printf(" %d ", frames[j]);
    } printf("\n"); } printf("Total Page
Faults: %d\n", faults);
}

```

```

void lru(int pages[], int n, int capacity) {
    int frames[capacity], recent[capacity], faults = 0;

    for (int i = 0; i < capacity; i++) frames[i] = -1;

    printf("\nLRU Page Replacement\n");
    for (int i = 0; i < n; i++) { int found = 0;
        for (int j = 0; j < capacity; j++) { if
            (frames[j] == pages[i]) { recent[j] = i;
                found =
                    1; break;
            }
        }

        if (!found) {
            int lru_index = 0; for (int j = 1; j < capacity; j++)
                { if (frames[j] == -1 || recent[j] <
                    recent[lru_index])
                        lru_index = j;
                }
            frames[lru_index] =

```



```

        pages[i]; recent[lru_index] = i;
        faults++;
    }

    printf("Frames: "); for (int j =
0; j < capacity; j++) { if
(frames[j] == -1) printf(" - ");
    else
        printf(" %d ", frames[j]);
    } printf("\n"); } printf("Total Page
Faults: %d\n", faults);
}

void optimal(int pages[], int n, int capacity) {
    int frames[capacity], faults = 0;

    for (int i = 0; i < capacity; i++) frames[i] = -1;

    printf("\nOptimal Page Replacement\n");
    for (int i = 0; i < n; i++) { int found = 0;
    for (int j = 0; j < capacity; j++) { if
(frames[j] == pages[i]) { found = 1; break;
        } } if
(!found) {    int opt_index = -1,
        farthest = i; for
        (int j = 0; j < capacity; j++) { if
            (frames[j] == -1) {
                opt_index = j; break;

```

```

    }
    int next_use = INT_MAX; for
    (int k = i + 1; k < n; k++) {
        if (frames[j] == pages[k]) {
            next_use = k;
            break;
        } } if (next_use > farthest) {
        farthest = next_use; opt_index
        = j;
    } } frames[opt_index] = pages[i];
    faults++;
}

```

```

printf("Frames: "); for (int j = 0; j
< capacity; j++) { if (frames[j]
== -1)

    printf(" - ");
else
    printf(" %d ", frames[j]);
} printf("\n"); } printf("Total Page
Faults: %d\n", faults);
}

```

```

int main() { int n, capacity; printf("Enter number
of pages: "); scanf("%d", &n); int pages[n];
printf("Enter the page reference string: "); for
(int i = 0; i < n; i++) scanf("%d", &pages[i]);
printf("Enter number of frames: ");

```

```
scanf("%d", &capacity);
```

```
fifo(pages, n, capacity); lru(pages, n,  
capacity);
```

```
optimal(pages,      n,  
capacity);
```

```
return 0;
```

```
}
```

Result:

```
Enter number of pages: 10
Enter the page reference string: 7 0 1 2 0 3 0 4 2 3
Enter number of frames: 3
```

FIFO Page Replacement

```
Frames: 7 - -
Frames: 7 0 -
Frames: 7 0 1
Frames: 2 0 1
Frames: 2 0 1
Frames: 2 3 1
Frames: 2 3 0
Frames: 4 3 0
Frames: 4 2 0
Frames: 4 2 3
Total Page Faults: 9
```

LRU Page Replacement

```
Frames: - - 7
Frames: - - 0
Frames: - - 1
Frames: - - 2
Frames: - - 0
Frames: - - 3
Frames: - - 0
Frames: - - 4
Frames: - - 2
Frames: - - 3
Total Page Faults: 10
```

Optimal Page Replacement

```
Frames: 7 - -
Frames: 7 0 -
Frames: 7 0 1
Frames: 2 0 1
Frames: 2 0 1
Frames: 2 0 3
Frames: 2 0 3
Frames: 2 4 3
Frames: 2 4 3
Frames: 2 4 3
Total Page Faults: 6
```

